

WEEK 9

Aim:

To develop a user interface for a real-world scenario using React (JSX – JavaScript Extension), namely Premium Fix, a home services booking application (Plumber, Home Cleaning, Laundry and Electrician), with a Registration Form with validation.

Description:

Premium Fix is a single-page React application built with Vite in which users book home services online. The page is divided into reusable function components: App (navigation, page switching and shared state), ServiceCard (one card per service), RegisterForm (registration with validation) and BookingForm (booking pop-up). The service details (name, icon, price, duration, description) are kept in data.js and displayed as cards using the map() function. The Registration Form collects full name, email, 10-digit mobile number, city, password, confirm password and a terms checkbox; the values are held in state with useState, and on submit a validate() function checks every field (name length, email pattern, mobile pattern, city selected, minimum password length, password match and terms accepted) and shows an error message under each wrong field. After successful registration the navbar greets the user by name. Clicking Book on a service opens a pop-up where the user picks the date, time slot and address; if the user has not registered, the pop-up asks to register first. A confirmed booking is added to the bookings state and listed under My bookings, from where it can be cancelled. Props such as onBook, onDone, onConfirm and onClose are used to pass data and actions between components.

Flow: Landing Page → Registration Form → Validate Fields → Select Service → Booking Form → Confirm Booking → My Bookings → Cancel.

Program:

Premium Fix – Home Services Booking Application Using React and JSX

Program Code:

package.json

```
{
  "name": "premium-fix",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": { "dev": "vite", "build": "vite build", "preview": "vite preview" },
  "dependencies": { "react": "^18.3.1", "react-dom": "^18.3.1" },
  "devDependencies": { "@vitejs/plugin-react": "^4.3.1", "vite": "^5.4.0" }
}
```

vite.config.js

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
export default defineConfig({ plugins: [react()] });
```

index.html (HTML entry page)

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Premium Fix - Home Services</title>
    <link
href="https://fonts.googleapis.com/css2?family=Bricolage+Grotesque:wght@600;800&family=Manrope:wght@400;600;700&display=swap" rel="stylesheet" />
    </head>
    <body><div id="root"></div><script type="module" src="/src/main.jsx"></script></body>
  </html>
```

src/main.jsx (React entry point)

```
import React from "react";
import { createRoot } from "react-dom/client";
import App from "../App.jsx";
import "../styles.css";
createRoot(document.getElementById("root")).render(<App />);
```

src/data.js (services data)

```
export const services = [
  { id: 1, name: "Plumber", icon: "🔧", price: 299, time: "60 min", desc: "Leaks, taps, blocked drains and pipe fitting." },
  { id: 2, name: "Home Cleaning", icon: "🧹", price: 799, time: "3 hrs", desc: "Deep clean for kitchen, bathrooms and rooms." },
  { id: 3, name: "Laundry", icon: "🧺", price: 149, time: "24 hrs", desc: "Wash, dry and fold with doorstep pickup." },
  { id: 4, name: "Electrician", icon: "💡", price: 249, time: "45 min", desc: "Switches, fans, wiring and appliance repair." },
];
export const slots = ["9:00 AM", "11:00 AM", "1:00 PM", "3:00 PM", "5:00 PM"];
```

src/App.jsx (main component)

```
import { useState } from "react";
import { services } from "../data";
```

```

import ServiceCard from "./components/ServiceCard";
import BookingForm from "./components/BookingForm";
import RegisterForm from "./components/RegisterForm";
export default function App() {
  const [page, setPage] = useState("home");
  const [selected, setSelected] = useState(null);
  const [user, setUser] = useState(null);
  const [bookings, setBookings] = useState([]);
  const addBooking = (b) => {
    setBookings([...bookings, { ...b, id: Date.now() }]);
    setSelected(null);
  };
  const cancel = (id) => setBookings(bookings.filter((b) => b.id !== id));
  return (
    <>
      <header className="nav">
        <h1 className="logo" onClick={() => setPage("home")}>Premium Fix</h1>
        <nav>
          <button className="link" onClick={() => setPage("home")}>Services</button>
          {user ? (
            <span className="hello">Hi, {user.name.split(" ")[0]}</span>
          ) : (
            <button className="btn small" onClick={() =>
setPage("register")}>Register</button>
          )}
        </nav>
      </header>
      {page === "register" ? (
        <RegisterForm onDone={(u) => { setUser(u); setPage("home"); }} />
      ) : (
        <main>
          <section className="hero">
            <h2>Something broke? Book a verified pro in minutes.</h2>
            <p>Plumbing, cleaning, laundry and electrical work at your door, with fixed
prices.</p>
          </section>
          <section className="grid">
            {services.map((s) => (
              <ServiceCard key={s.id} service={s} onBook={() => setSelected(s)} />
            ))}
          </section>
          <section className="mybookings">
            <h3>My bookings</h3>
            {bookings.length === 0 ? (
              <p className="muted">Nothing booked yet. Pick a service above to get
started.</p>
            ) : (
              bookings.map((b) => (
                <BookingForm key={b.id} booking={b} onBook={() => setSelected(b)} />
              ))
            )}
          </section>
        </main>
      )}
    </>
  );
}

```

```

    ): (
      bookings.map((b) => (
        <div className="row" key={b.id}>
          <div>
            <strong>{b.service}</strong>
            <div className="muted">{b.date} at {b.slot} · {b.address}</div>
          </div>
          <button className="link danger" onClick={() => cancel(b.id)}>Cancel</button>
        </div>
      ))
    )}
  </section>
</main>
)}
{selected && (
  <BookingForm
    service={selected}
    user={user}
    onClose={() => setSelected(null)}
    onRegister={() => { setSelected(null); setPage("register"); }}
    onConfirm={addBooking}
  />
)}
</>
);
}

```

src/components/ServiceCard.jsx

```

export default function ServiceCard({ service, onBook }) {
  return (
    <article className="card">
      <div className="icon">{service.icon}</div>
      <h3>{service.name}</h3>
      <p className="muted">{service.desc}</p>
      <div className="meta">
        <span>From ₹{service.price}</span>
        <span>{service.time}</span>
      </div>
      <button className="btn" onClick={onBook}>Book {service.name}</button>
    </article>
  );
}

```

src/components/BookingForm.jsx (booking form)

```

import { useState } from "react";

```

```

import { slots } from "../data";
export default function BookingForm({ service, user, onClose, onRegister, onConfirm }) {
  const [f, setF] = useState({ date: "", slot: slots[0], address: "" });
  const [err, setErr] = useState("");
  const set = (k) => (e) => setF({ ...f, [k]: e.target.value });
  const submit = (e) => {
    e.preventDefault();
    if (!f.date || f.address.trim().length < 6) {
      setErr("Choose a date and enter your full address.");
      return;
    }
    onConfirm({ ...f, service: service.name });
  };
  return (
    <div className="overlay" onClick={onClose}>
      <div className="modal" onClick={(e) => e.stopPropagation()}>
        <h3>Book {service.name}</h3>
        {!user ? (
          <div>
            <p className="muted">Create an account first so we can confirm your booking.</p>
            <button className="btn" onClick={onRegister}>Register now</button>
          </div>
        ) : (
          <form onSubmit={submit} noValidate>
            <label>Date
              <input type="date" value={f.date} min={new Date().toISOString().split("T")[0]}
onChange={set("date")} />
            </label>
            <label>Time slot
              <select value={f.slot} onChange={set("slot")}>
                {slots.map((s) => <option key={s}>{s}</option>)}
              </select>
            </label>
            <label>Address
              <textarea rows="2" value={f.address} onChange={set("address")}
placeholder="House no, street, area" />
            </label>
            {err && <p className="error">{err}</p>}
            <button className="btn" type="submit">Confirm booking</button>
          </form>
        )}
        <button className="link" onClick={onClose}>Close</button>
      </div>
    </div>
  );
}

```

src/components/RegisterForm.jsx (Registration Form)

```
import { useState } from "react";
const empty = { name: "", email: "", phone: "", city: "", password: "", confirm: "", terms: false };
export default function RegisterForm({ onDone }) {
  const [f, setF] = useState(empty);
  const [errors, setErrors] = useState({});
  const change = (e) => {
    const { name, value, type, checked } = e.target;
    setF({ ...f, [name]: type === "checkbox" ? checked : value });
  };
  const validate = () => {
    const e = {};
    if (f.name.trim().length < 3) e.name = "Enter your full name (min 3 letters).";
    if (!/^\S+@\S+\.\S+$/.test(f.email)) e.email = "Enter a valid email address.";
    if (!/^[6-9]\d{9}$/.test(f.phone)) e.phone = "Enter a 10-digit mobile number.";
    if (!f.city) e.city = "Select your city.";
    if (f.password.length < 6) e.password = "Password must be at least 6 characters.";
    if (f.confirm !== f.password) e.confirm = "Passwords do not match.";
    if (!f.terms) e.terms = "Accept the terms to continue.";
    return e;
  };
  const submit = (ev) => {
    ev.preventDefault();
    const e = validate();
    setErrors(e);
    if (Object.keys(e).length === 0) onDone(f);
  };
  const Field = ({ name, label, type = "text", ...rest }) => (
    <label>{label}
      <input name={name} type={type} value={f[name]} onChange={change} {...rest} />
      {errors[name] && <span className="error">{errors[name]}</span>}
    </label>
  );
  return (
    <main className="auth">
      <form className="panel" onSubmit={submit} noValidate>
        <h2>Create your account</h2>
        <p className="muted">One account to book, track and cancel services.</p>
        <Field name="name" label="Full name" />
        <Field name="email" label="Email" type="email" />
        <Field name="phone" label="Mobile number" type="tel" maxLength={10} />
        <label>City
          <select name="city" value={f.city} onChange={change}>
            <option value="">Select city</option>
          </select>
        </label>
      </form>
    </main>
  );
}
```

```

        [{"Hyderabad", "Bengaluru", "Chennai", "Mumbai", "Delhi"].map((c) => <option
key={c}>{c}</option>)}
    </select>
    {errors.city && <span className="error">{errors.city}</span>}
  </label>
  <Field name="password" label="Password" type="password" />
  <Field name="confirm" label="Confirm password" type="password" />
  <label className="check">
    <input type="checkbox" name="terms" checked={f.terms} onChange={change} />
    I agree to the terms and privacy policy
  </label>
  {errors.terms && <span className="error">{errors.terms}</span>}
  <button className="btn" type="submit">Create account</button>
</form>
</main>
);
}

```

src/styles.css

```

:root { --ink:#0F2A43; --bg:#F2F6F9; --card:#fff; --gold:#F2B134; --teal:#1F8A70; --
muted:#5B6B7B; --red:#C0392B; }
* { box-sizing: border-box; }
body { margin:0; font-family:"Manrope",sans-serif; background:var(--bg); color:var(--ink); }
h1,h2,h3 { font-family:"Bricolage Grotesque",sans-serif; margin:0; }
.nav { display:flex; justify-content:space-between; align-items:center; padding:16px 6vw;
background:var(--ink); color:#fff; position:sticky; top:0; z-index:5; }
.nav nav { display:flex; align-items:center; gap:16px; }
.logo { font-size:1.5rem; cursor:pointer; }
.logo::after { content:""; display:inline-block; width:10px; height:10px; background:var(--
gold); border-radius:50%; margin-left:6px; }
.hello { color:var(--gold); font-weight:700; }
main { padding:0 6vw 60px; }
.hero { padding:56px 0 32px; max-width:640px; }
.hero h2 { font-size:clamp(1.9rem,5vw,3rem); line-height:1.1; }
.hero p { color:var(--muted); font-size:1.05rem; }
.grid { display:grid; grid-template-columns:repeat(auto-fit,minmax(230px,1fr)); gap:20px; }
.card { background:var(--card); border-radius:16px; padding:22px; display:flex; flex-
direction:column; gap:10px; border:1px solid #dde6ee; }
.icon { font-size:2rem; width:56px; height:56px; display:grid; place-items:center;
background:#FFF4D9; border-radius:14px; }
.meta { display:flex; justify-content:space-between; font-weight:700; color:var(--teal);
margin-top:auto; }
.muted { color:var(--muted); margin:0; font-size:.92rem; }
.btn { background:var(--gold); color:var(--ink); border:0; padding:12px 18px; border-
radius:10px; font:700 .95rem "Manrope"; cursor:pointer; }
.btn:hover { filter:brightness(.95); }

```

```

.btn.small { padding:8px 14px; }
.link { background:none; border:0; color:inherit; font:600 .95rem "Manrope"; cursor:pointer;
}
.link.danger { color:var(--red); }
:focus-visible { outline:3px solid var(--teal); outline-offset:2px; }
.mybookings { margin-top:44px; }
.row { display:flex; justify-content:space-between; align-items:center; background:var(--card); padding:14px 18px; border-radius:12px; margin-top:10px; border-left:5px solid var(--teal); }
.overlay { position:fixed; inset:0; background:rgba(15,42,67,.6); display:grid; place-items:center; padding:16px; z-index:10; }
.modal, .panel { background:#fff; border-radius:18px; padding:26px; width:100%; max-width:440px; display:flex; flex-direction:column; gap:14px; }
.modal form { display:flex; flex-direction:column; gap:14px; }
.auth { display:grid; place-items:center; padding-top:40px; }
label { display:flex; flex-direction:column; gap:6px; font-weight:600; font-size:.9rem; }
input, select, textarea { padding:11px 12px; border:1.5px solid #cbd7e2; border-radius:10px; font:inherit; background:#fff; }
input:focus, select:focus, textarea:focus { border-color:var(--teal); outline:none; }
.check { flex-direction:row; align-items:center; gap:10px; font-weight:400; }
.check input { width:18px; height:18px; }
.error { color:var(--red); font-size:.82rem; font-weight:600; margin:0; }

```

Execution of the Program:

Method	Purpose
React Component	Creates the Premium Fix UI using App, ServiceCard, RegisterForm and BookingForm
useState()	Stores the current page, selected service, user, bookings and form values
map()	Displays every service from data.js as a card
validate()	Checks every registration field and shows an error under each wrong field
Props (onBook, onDone, onConfirm, onClose)	Pass data and actions between components
Conditional Rendering	Shows the Register button or user greeting, the booking pop-up and the My bookings list

Details about the Program:

Where Saved	All files are saved inside the project folder FL_LAB_ANITS/fs_week_9/ in the GitHub repository raviteja-codehub/FL_LAB_ANITS — package.json, vite.config.js, index.html and the src/ folder with main.jsx, App.jsx, data.js, styles.css and the components/ folder (ServiceCard.jsx, BookingForm.jsx, RegisterForm.jsx). node_modules is not uploaded.
Where Executed	The application runs on the Vite development server (Node.js) from the terminal on the local computer and is opened in a web browser (Google Chrome / Edge / Firefox) at http://localhost:5173 .
How Executed	1. Open a terminal in the fs_week_9 folder. 2. Run npm install to install React and Vite. 3. Start the app with npm run dev; the terminal prints the local URL http://localhost:5173/. 4. Open the URL in the browser, click Register, fill the form and create the account, choose a service, book it and view it under My bookings. Use Cancel to remove a booking.

Working:

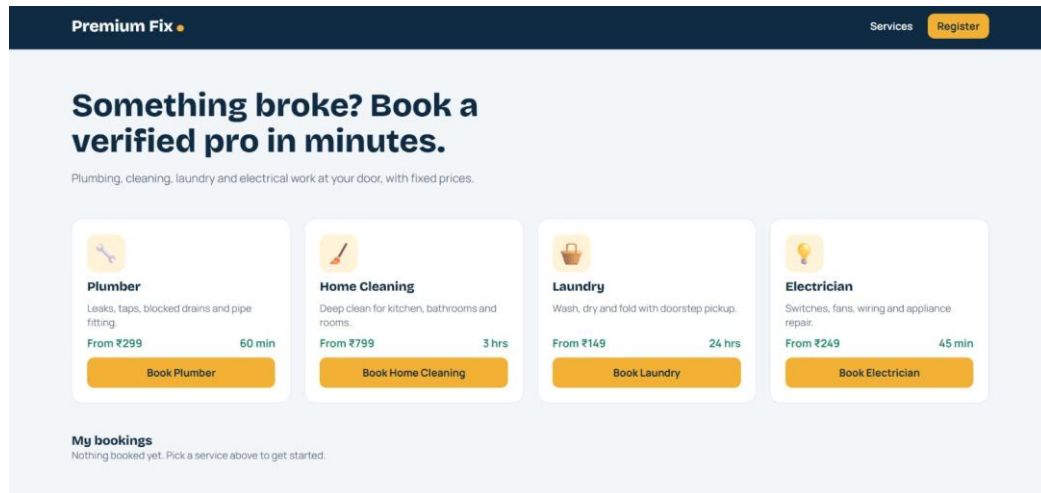
1. The React application displays the Premium Fix landing page with the four services (Plumber, Home Cleaning, Laundry and Electrician) as cards created using map().
2. The user clicks Register and the Registration Form opens with Full Name, Email, Mobile Number, City, Password, Confirm Password and Terms fields.
3. The entered values are stored in the form state using useState and updated by the change() handler.
4. When Create account is clicked, preventDefault() stops the page refresh and the validate() function checks every field.
5. If any field is wrong, an error message is shown under that field; otherwise the user is saved and the navbar greets the user by name.
6. The user clicks Book on a service and the booking pop-up opens. If the user is not registered, the pop-up asks to register first.
7. The user selects the date, time slot and address and clicks Confirm booking.
8. The confirmed booking is added to the bookings state and listed under My bookings, from where it can be removed using Cancel.

Output:

After running npm run dev and opening <http://localhost:5173>, the landing page, registration form and booking pages appear as shown below:

Output Screenshots:

Output 1: Landing Page (services list)



Output 2: Registration Page (Registration Form)

The screenshot shows the registration form on the Premium Fix website. The header is dark blue with the logo 'Premium Fix' and a 'Services' link. A yellow 'Register' button is in the top right. The form is titled 'Create your account' with a subtext: 'One account to book, track and cancel services.' The form fields are: 'Full name' (Raviteja Pasupuleti), 'Email' (ravi@gmail.com), 'Mobile number' (8886898943), 'City' (Hyderabad), 'Password' (masked with asterisks), and 'Confirm password' (masked with asterisks). There is a checkbox for 'I agree to the terms and privacy policy' which is checked. A yellow 'Create account' button is at the bottom.

Output 3: Service Booking Form (Electrician)

The screenshot shows the 'Premium Fix' website with a dark blue header. The main heading is 'Something broke? Book a verified pro in minutes.' Below it, a subheading reads 'Plumbing, cleaning, laundry and electrical work at your door, with fixed prices.' A modal form titled 'Book Electrician' is open in the center. It contains the following fields: 'Date' (10/25/2026), 'Time slot' (9:00 AM), and 'Address' (2/43, ANITS Road, Vizag). At the bottom of the modal are 'Confirm booking' and 'Close' buttons. In the background, service cards for 'Plumber', 'Home Cleaning', and 'Electrician' are visible, each with an icon, description, price, and duration.

Output 4: My Bookings (after confirming the booking)

The screenshot shows the 'Premium Fix' website after a booking. The main heading is 'Something broke? Book a verified pro in minutes.' Below it, a subheading reads 'Plumbing, cleaning, laundry and electrical work at your door, with fixed prices.' Four service cards are displayed: 'Plumber', 'Home Cleaning', 'Laundry', and 'Electrician'. Each card has an icon, description, price, and duration. Below the service cards is a 'My bookings' section. It shows a single booking for 'Electrician' on '2025-10-25 at 9:00 AM' at the address '2/43, ANITS Road, Vizag'. A 'Cancel' button is visible next to the booking details.

Result:

The **Premium Fix home services booking UI** was successfully developed using **React and JSX**. The application demonstrates the use of JSX, reusable function components, **useState**, props, the **map()** function, controlled form inputs, form validation using **validate()**, event handling with **preventDefault()**, and conditional rendering to dynamically display the registration, booking and My bookings details.